

Deployment of Vertical Pod Autoscaler (VPA) in Kubernetes

Overview

The Vertical Pod Autoscaler (VPA) automatically adjusts CPU and memory requests/limits for pods based on actual resource usage. It helps optimize resource utilization and reduces manual tuning effort.

Before deploying VPA, ensure that the Kubernetes Metrics Server is installed and functioning correctly, since VPA depends on resource metrics collection.

Prerequisites

Verify Metrics Server Installation

Run the following command:

```
kubectl get deployment metrics-server -n kube-system
```

Expected output:

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
metrics-server	1/1	1	1	10d

You can also validate metrics collection using:

```
kubectl top nodes
```

or

```
kubectl top pods -A
```

If metrics are displayed successfully, the Metrics Server is working correctly.

Step 1: Navigate to Your Working Directory

Move to the directory where you want to download the VPA source code.

```
cd /path/to/your/directory
```

Example:

```
cd /home/ubuntu
```

Step 2: Clone the Kubernetes Autoscaler Repository

Clone the official Kubernetes Autoscaler repository from GitHub.

```
git clone https://github.com/kubernetes/autoscaler.git
```

This downloads the latest source code for the Vertical Pod Autoscaler.

Step 3: Navigate to the VPA Directory

Change to the Vertical Pod Autoscaler directory.

```
cd autoscaler/vertical-pod-autoscaler/
```

You are now inside the VPA source directory.

Step 4: Deploy the VPA Components

Run the deployment script:

```
./hack/vpa-up.sh
```

This script automatically deploys all required VPA components into the Kubernetes cluster.

What the `vpa-up.sh` Script Does

The script performs several important tasks automatically:

1. Creates Custom Resource Definitions (CRDs)

CRDs are required to define the `VerticalPodAutoscaler` custom resource in Kubernetes.

2. Configures RBAC Permissions

Role-Based Access Control (RBAC) permissions are created to allow VPA components to securely interact with the cluster.

3. Deploys VPA Components

The following components are deployed:

Component	Purpose
Recommender	Analyzes usage metrics and suggests resource recommendations
Updater	Evicts pods when resource updates are needed
Admission Controller	Mutates pod resource requests during pod creation

These components are deployed into the kube-system namespace.

Step 5: Validate the VPA Deployment

Check whether all VPA pods are running successfully.

```
kubectl get pods -n kube-system | grep vpa
```

Expected output:

```
vpa-admission-controller-xxxxx 1/1 Running
vpa-recommender-xxxxx          1/1 Running
vpa-updater-xxxxx              1/1 Running
```

All pods should be in the Running state.

Step 6: Create a VPA Object

Create a YAML file named vpa.yaml.

Example configuration:

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: stress-vpa
  namespace: default

spec:
  targetRef:
    apiVersion: apps/v1
```

```

kind: Deployment
name: high-cpu-utilization-deployment

updatePolicy:
  updateMode: Auto

resourcePolicy:
  containerPolicies:
    - containerName: "*"

    minAllowed:
      cpu: 100m
      memory: 50Mi

    maxAllowed:
      cpu: 200m
      memory: 500Mi

    controlledResources:
      - cpu
      - memory

```

Explanation of Important Fields

Field	Description
targetRef	Specifies the deployment managed by VPA
updateMode: Auto	Automatically updates pod resources
minAllowed	Minimum resources VPA can assign
maxAllowed	Maximum resources VPA can assign
controlledResources	Resources managed by VPA

Step 7: Apply the VPA Configuration

Deploy the VPA object:

```
kubectl apply -f vpa.yaml
```

Expected output:

```
verticalpodautoscaler.autoscaling.k8s.io/stress-vpa created
```

Step 8: Verify the VPA Object

Check the created VPA resource:

```
kubectl get vpa -n default
```

Expected output:

NAME	MODE	CPU	MEM	PROVIDED	AGE
stress-vpa	Auto	100m	50Mi	True	1m

Step 9: Describe the VPA for Recommendations

View detailed recommendations:

```
kubectl describe vpa stress-vpa -n default
```

You will see:

- Current recommendations
- Target CPU/memory
- Lower and upper bounds
- Container-specific recommendations

Example:

Recommendation:

```
Container Recommendations:
  Container Name:  app-container
  Target:
    Cpu:          150m
    Memory:       262144k
```

Important Notes

VPA Modes

Mode	Description
Off	Only provides recommendations
Initial	Applies recommendations only during pod creation
Auto	Automatically updates running pods
Recreate	Recreates pods to apply new values

Limitation of VPA with HPA

Avoid using VPA and Horizontal Pod Autoscaler (HPA) together on the same CPU/memory metrics because they may conflict.

Recommended approach:

- Use VPA for resource optimization
 - Use HPA with custom/external metrics if required
-

Delete VPA Components

Delete the VPA Object

```
kubectl delete -f vpa.yaml
```

Remove VPA Components Completely

```
./hack/vpa-down.sh
```

Useful Commands

Check VPA Resources

```
kubectl get vpa -A
```

Check Recommendations

```
kubectl describe vpa <vpa-name> -n <namespace>
```

Check Metrics

```
kubectl top pods  
kubectl top nodes
```

Check VPA Logs

```
kubectl logs -n kube-system deployment/vpa-recommender  
kubectl logs -n kube-system deployment/vpa-updater  
kubectl logs -n kube-system deployment/vpa-admission-controller
```